面向对象题型整理

改错题

- ```
  int *zPtr;
  int z[5] = {1, 2, 3, 4, 5};

  zPtr = ++z;//错误
  ```

  数组名不可自增++,数组名不是可修改的左值

  1. `zPtr = z + 1`
  2. `*zPtr = z`
  3. `zPtr ++`
- student s2 = 2;等价于student s2(2);
- 被声明为const int 常量形参不能被修改
- ```
  class C {
      int& c;
  public:
      C() { c = 0;//错误 }
  };
  ```

  - 引用成员必须在构造函数初始化列表中初始化
  - 不能在构造函数体内给引用赋值

  ```
  C(int &x):c(x){}
  C():c(0){}
  ```

- `int &ref1 = 10;` 错误！非常量引用不能绑定字面量

  `const int&ref2 = 10;` 正确！常量引用可以绑定到字面量

- ```
  namespace mySpace {
      float x;
  }

  namespace mySpace2 {
      int i;
      float x;
  }

  using mySpace::x = 1;//错误
  ```

  - using用途

    > 引入整个命名空间 `using namespace mySpace;` 之后可以用x

    > 引入某个名字（变量/函数/类型） `using mySpace::x;` 之后可以直接写x

  - using是声明语句,只能声明名字，不能改值
  - 如果想给x赋值，应该写 `mySpace::x = 1;` 这是对命名空间变量赋值的唯一合法方式
  ```

- 如果using写在函数内部

```
void f() {
    using mySpace::x;
    x = 5;      // OK
}

int main() {
    x = 3;      // ❌ 错误：这里没有 using
}
```

-
```
template <class T>
class myTemp {
public:
    void m();
};

void myTemp::m() {
    /* m's body */
}
```

模版函数正确写法 **模版类的成员函数在类外定义时，必须指明模版参数T**

```
template <class T>
void myTemp<T>::m() {
    /* m's body */
}
```

  - 声明这是一个模版函数
  - 明确指出：这是`mtTemp<T>`这个模版实例的成员
- 模版类在类外声明对象`myTemp a;`错

`myTemp<int> a;myTemp<double> b;`正确，必须声明模板参数

-
```
class MyClass {
private:
    int x;
public:
    MyClass(int val) : x(val)
    {
    }
    void set(int i) { x = i;}
    int get() { return x;}
};

int main() {
    const MyClass foo(10);//错误
    foo.set(20);

    return 0;
}
```

const MyClass 创建了一个常量对象

  - const对象只能调用const成员函数，不能调用非const成员函数
  - const对象的所有成员变量默认都是const的

- 在类内定义的非const变量x，类外定义const对象可以对x进行访问，但是不能修改
- 构造函数和析构函数是例外，即使对象时const也能调用

> const成员函数承诺不修改对象状态 `int get()const{return x;}`

- `catch(...)`可以捕获所有异常，但无法获取异常对象信息，就是只能知道**发生了异常，只能写在所有catch的最后**，否则程序直接不合法
-
```
class BC {
public:
    BC(int a) { x = a; z = -1;}
    BC(int a1, int a2) {
        x = a1; z = a2;
    }
private:
    int x, z;
};

class DC : public BC {
public:
    DC(int a) { y = a;}  // 错误：没有调用基类构造函数
private:
    int y;
};
```

  - DC类的构造函数没有显示调用基类BC的构造函数
  - 由于BC没有默认构造函数，编译器无法自动调用

```
class DC : public BC {
public:
    DC(int a) : BC(a), y(a) {}  // 显式调用基类构造函数
private:
    int y;
};
```

  - 构造函数调用顺序按继承顺序
  - 派生类不继承基类的构造函数
  - 如果基类没有默认构造函数，派生类必须**显示调用**某个基类构造函数`DC():BC(){}`
- 构造函数不能有返回类型(包括void)
- 如何判断一个函数是构造函数
  - 与类同名
  - 没有返回类型
  - 可以有参数包括默认参数
  - 可以是public、protected、private
- 默认构造函数

  `MyClass();`

  `MyClass(int x = 0);`
- 拷贝构造函数

  `MyClass(const MyClass& other);`

  `MyClass(MyClass& other);`

  `MyClass(MyClass other);`不是，参数不能传值，会导致无限递归

- 模板构造函数

```
template<typename U>
Container(const Container<U>&)
```

- 未指定访问权限默认私有
- 可以访问私有成员的有：
  - 类自身的成员函数
  - 类自身的友元函数
  - 类自身的友元类
- 不能访问私有成员的有：
  - 类外普通函数
  - 其他类的成员函数
  - 派生类的成员函数 子类不能访问父类私有成员
  - 类外对象不能直接访问
-

```
class C {
    int sz;
public:
    friend C operator+ (const C&, const C&);
    //...
};

C C::operator+ (const C& c1, const C& c2){//错误
    //...
}
```

  - operator+被声明为友元函数(friend)
  - 友元函数不是类的成员函数，所以不能使用C::作用域限定符
  - 友元函数是全局函数，应该在类外直接定义

`C operator+();`

-

```
class C {
public:
    void m() {/* ... */ }
    static void s() {/* ... */ }
};
void main() {
    C c1;
    c1.m();
    C::m();    // 错误在这里
}
```

  - m()是非静态成员函数，必须通过对象调用
  - 只有静态成员函数才能用`类名::函数名（）`的方式调用
- c++中成员函数有两种：
  1. 静态成员函数(static)
     - 属于类本身，不依赖具体对象
     - 可以直接通过类名调用 C::m();
     - 不能访问非静态成员变量，因为没有具体对象
  2. 非静态成员函数
     - 属于类的对象，必须通过对象调用c.m();

- ```
  template <class T, int i> class Array {
      int sz;
  public:
      Array():sz(i) { }
      //...
  };
  void f(int x) {
      Array<int, x> ay;     // 错误
  }
  ```

  - 模版的非类型参数(如int i)必须是编译时常量
  - x是函数参数，是运行时变量，值在编译时无法确定

  ```
  // 方法1：使用常量
  void f() {
      const int size = 10;
      Array<int, size> ay;  // 正确：size是编译时常量
  }

  // 方法2：使用constexpr（C++11）
  void f(constexpr int x) {  // 如果x是constexpr
      Array<int, x> ay;      // 正确
  }

  // 方法3：使用常量表达式
  void f() {
      Array<int, 5+3> ay;    // 正确：常量表达式
  }
  ```

- ```
  float* ptr = new float[20];
  for (int i = 0; i < 20; i++)
      ptr[i] = i + 2;
  delete ptr;//错误
  ```

  - ptr使用new[]申请的数组，必须用delete[]释放

- ```
  namespace myspace {
      void do() { /* ... */ }
      int temp;
  }
  using namespace myspace;
  using namespace myspace::temp;//错误
  ```

  - temp是变量不是命名空间，而不是命名空间本身，他是一个int类型变量
  - using namespace只能用在“命名空间名”
  - 想用temp，应该`using myspace::temp;`
  ```

- ```
  class C {
      int x;
      void setx(int a) { /* ... */ }
  };
  void main() {
      C c1;
      c1.setx(3);//错误
  }
  ```
  - setx是私有成员函数(class默认)，不能在类外调用
- ```
  class Student {
  public:
      void Student();
      ~Student();
  };
  ```
  - 构造函数不能有返回值，哪怕是void
- ```
  class BC {
      int x;
  public:
      BC(int xx = 0) { x = xx; }
  };

  class DC : public BC {
      char c;
  public:
      DC(int x1, char c1) {
          x = x1;//错误 这里错误的原因是派生类不能访问基类成员 而不是需要初始化列表
          c = c1;
      }
  };
  ```
  - x是基类BC的私有成员函数，派生类DC不能直接访问
  - 必须构造函数初始化基类`DC(int x1,char c1):BC(x1),c(c1){}`

  1. 构造顺序：先构造基类->再构造派生类
  2. 基类私有成员：
     - 派生类不可访问
     - 必须通过初始化列表调用基类构造函数
  3. 基类非私有成员
     - 派生类可在构造函数体内赋值
     - 不强制使用初始化列表
  4. 必须使用初始化，列表的情况
     - const成员
     - 引用成员
     - 基类无默认构造函数
     - 需要显式调用带参基类构造函数
  ```

```
class C {
    int sz;
public:
    friend C operator+(const C&, const C&);
};

C C::operator+(const C& c1, const C& c2) {
    cout << c1.sz;
}
```

- 这里声明的友元是非成员函数，不能用C::作用域限定
- 正确写法

  ```
  C operator+(const C& c1,const C* c2){
      cout<<c1.sz;
  }
  ```

- ```
  Class B{
  public:
      B(int a = 10,float y){i=a;z=y;}
  private:
      int i;
      float z;
  };
  ```

  - 有默认值的参数必须在参数列表末尾，当一个参数有默认值时，它后面的所有参数也必须有默认值

- ```
  class Animal{
  public:
      virtual void f() = 0;//有纯虚函数 Animal是抽象类
  };
  void f(){
      Animal B;//不能直接创建对象
  }
  ```

  - 抽象类
    - 包含至少一个纯虚函数 `virtual void f() = 0;`
    - 不能直接实例化
  - 纯虚函数
    - 语法： `virtual 返回类型 函数名(参数) = 0`
    - 派生类必须实现它 除非派生类也是抽象类

- ```
  class C {
  public:
      void m();//m是普通成员函数  不能C::m()
      static void s();//静态成员函数 可以C::s
  };

  void main() {
      C c1;
      c1.m();   // ✓
      c1.s();   // ✓
      C::m();   // ✗
      C::s();   // ✓
  }
  ```
  - 非静态成员函数不能通过类名直接调用
- ```
  template <class T, int i>
  class Array {
      T sz;
  public:
      Array(): sz(i) {}
      void f(int x)//错误 {
          Array<int, x> ay;
      }
  };
  ```
  - 非类型模版参数必须是编译期常量，这里的x是运行时变量